

# **Laporan Praktikum PBO**

## ***Jobsheet W03 - Enkapsulasi***



Oleh:

Muhammad Ramdhani Ariyadin (254107060018)

SIB 2-E

Link Github : <https://github.com/MuhammadRamdhaniAriyadin/PBO>

## Percobaan 1 constructor

1. Apa fungsi dari constructor?
2. Sebutkan keistimewaan constructor dibanding method lain pada suatu class.
3. Lakukan analisa dan buatlah kesimpulan, apakah constructor bisa memiliki access level modifier private?

Jawaban

1. untuk menginisialisasi nilai awal bagi atribut-atribut dari objek yang baru dibuat tersebut.
2. Keistimewaannya :
  - Nama yang Sama: Nama constructor harus persis sama dengan nama class-nya
  - Tidak Memiliki Return Type: Constructor tidak memiliki return type (bahkan void sekalipun tidak dituliskan).
  - Dipanggil Otomatis: Dipanggil secara otomatis hanya satu kali ketika instansiasi objek dilakukan.
  - Default Constructor: Jika sebuah class tidak didefinisikan constructor-nya secara eksplisit, Java compiler akan secara otomatis menyediakan default constructor tanpa parameter.
3. Analisa: Constructor dapat diberi access level modifier private. Ketika constructor diberi modifier private, objek dari class tersebut tidak dapat diinstansiasi secara langsung dari luar class menggunakan keyword new.

Kesimpulan: Constructor private biasanya digunakan pada pola perancangan tertentu (seperti Singleton Pattern atau Utility Class yang hanya berisi static methods/variables), di mana pembuat objek dibatasi atau dikontrol penuh hanya dari dalam class itu sendiri.

## Percobaan 2 enkapsulasi

### Pertanyaan

1. Pada class MotorDemo, saat kita menambah kecepatan untuk pertama kalinya, mengapa muncul peringatan “Kecepatan tidak boleh lebih dari 0 jika mesin off”?
2. Mengapa atribut merek, kecepatan, dan statusMesin sebaiknya diset private?
3. Apa fungsi dari setter dan getter?
4. Modifikasi class Motor sehingga kecepatan maksimalnya adalah 100
5. Modifikasi class Motor sehingga kecepatan nya tidak boleh bernilai negatif

### Jawaban

1. Peringatan tersebut muncul karena secara default atribut statusMesin bernilai false (Off) saat objek diinstansiasi. Pada method setKecepatan(), terdapat logika validasi: `if (!this.statusMesin && kecepatan > 0)`. Karena statusMesin masih false dan nilai yang di-pass pada setKecepatan(50) lebih dari 0, maka kondisi if terpenuhi sehingga pesan peringatan dicetak dan nilai atribut kecepatan tidak diubah.
2. Atribut diset private untuk menerapkan prinsip information hiding / data hiding. Hal ini bertujuan untuk:
  - Mencegah Akses Langsung: Mencegah pihak luar / class lain mengubah nilai atribut secara acak tanpa melalui aturan/validasi (bypassing validation logic).
  - Menjaga Integritas Data: Memastikan data objek selalu valid (misalnya mencegah nilai kecepatan berubah menjadi negatif atau bergerak saat mesin dalam keadaan off).
  - Kemudahan Maintenance: Jika aturan bisnis berubah di kemudian hari, kita cukup mengubah kode di dalam method getter/setter pada class tersebut tanpa harus mengubah seluruh kode sistem.
3. Fungsi getter dan setter
  - Getter: Method yang berfungsi untuk membaca/mengembalikan nilai dari atribut bertipe private (memiliki return type dan tanpa parameter)
  - Setter: Method yang berfungsi untuk menetapkan, memanipulasi, atau mengupdate nilai dari atribut bertipe private (biasanya bertipe void dan memiliki parameter). Setter juga berperan sebagai "gerbang kontrol" penyaring validasi sebelum data dimasukkan ke atribut.
4. Modifikasi kecepatan maksimal 100  
Menambahkan kondisi pengecekan else if (`kecepatan > 100`). Jika nilai yang dimasukkan melebihi 100, program memberikan pesan peringatan dan secara otomatis menetapkan kecepatan ke nilai batas maksimum yaitu 100

```
else if (kecepatan > 100) {  
    System.out.println(x: "Kecepatan tidak boleh melebihi 100! Kecepatan diset ke 100.");  
    this.kecepatan = 100;  
}
```

Menguji kecepatan di atas 100

```
Motor motor3 = new Motor();
motor3.setPlatNomor(platNomor: "D 8343 CV");
motor3.setStatusMesin(statusMesin: true);
motor3.setKecepatan(kecepatan: 120);
motor3.displayStatus();
```

Output

```
=====
Kecepatan tidak boleh melebihi 100! Kecepatan diset ke 100.
Plat Nomor: D 8343 CV
Status Mesin: On
Kecepatan: 100
=====
```

5. Modifikasi kecepatan tidak boleh negatif  
Ditambahkan kondisi else if ( $\text{kecepatan} < 0$ ). Jika dimasukkan angka di bawah 0, program menolak input dan menampilkan pesan bahwa nilai kecepatan tidak boleh negatif

```
else if (kecepatan < 0) {
    System.out.println(x: "Kecepatan tidak boleh bernilai negatif!");
}
```

Menguji kecepatan negatif

```
// Menguji kecepatan di atas 100
Motor motor3 = new Motor();
motor3.setPlatNomor(platNomor: "D 8343 CV");
motor3.setStatusMesin(statusMesin: true);
motor3.setKecepatan(kecepatan: 120);
motor3.displayStatus();

// Menguji kecepatan negatif
motor3.setKecepatan(-20);
motor3.displayStatus();
```

Output

```
=====
Kecepatan tidak boleh melebihi 100! Kecepatan diset ke 100.
Plat Nomor: D 8343 CV
Status Mesin: On
Kecepatan: 100
=====
Kecepatan tidak boleh bernilai negatif!
Plat Nomor: D 8343 CV
Status Mesin: On
Kecepatan: 100
=====

d:\POLINEMA3\PBO>
```

## Tugas

1. Pada praktikum ini, diimplementasikan konsep enkapsulasi pada kelas Anggota untuk mengelola data dan operasi transaksi pinjaman pada sistem koperasi simpan pinjam
  - Penerapan Enkapsulasi: Seluruh atribut (nomorKtp, nama, limitPinjaman, jumlahPinjaman) diset bertipe private agar tidak dapat diubah secara acak dari luar kelas
  - Akses & Validasi Data: Nilai atribut diakses melalui method getter dan setter. Perubahan saldo pinjaman dikontrol secara ketat melalui method pinjam() dan angsur():
    - a. Method pinjam() memvalidasi agar total pinjaman tidak melebihi limitPinjaman
    - b. Method angsur() mengurangi jumlahPinjaman dan meyakinkan saldo pinjaman tidak bernilai negatif.

### Langkah langkah

#### a. Membuat class Anggota.java

- Deklarasikan atribut bertipe private: nomorKtp, nama, limitPinjaman, dan jumlahPinjaman

```
prikum03 > J Anggota.java > Anggota > Ang
public class Anggota {
    private String nomorKtp;
    private String nama;
    private int limitPinjaman;
    private int jumlahPinjaman;
```

- Buat constructor untuk menginisialisasi nomorKtp, nama, dan limitPinjaman, serta set jumlahPinjaman = 0 secara default.

```
public Anggota(String nomorKtp, String nama, int limitPinjaman) {
    this.nomorKtp = nomorKtp;
    this.nama = nama;
    this.limitPinjaman = limitPinjaman;
    this.jumlahPinjaman = 0; // awal jumlah pinjaman
}
```

- Buat method getter untuk seluruh atribut dan setter untuk nama

```
}
public String getNama() {
    return nama;
}
public void setNama(String nama) {
    this.nama = nama;
}
public String getNomorKtp() {
    return nomorKtp;
}
public int getLimitPinjaman() {
    return limitPinjaman;
}
public int getJumlahPinjaman() {
    return jumlahPinjaman;
}
```

- Tambahkan method pinjam(int nominal) untuk mengecek batasan limit.

```
public void pinjam(int nominal) {
    if ((jumlahPinjaman + nominal) > limitPinjaman) {
        System.out.println(x: "Maaf, jumlah pinjaman melebihi limit.");
    } else {
        jumlahPinjaman += nominal;
    }
}
```

- Tambahkan method angsur(int nominal) untuk mengurangi jumlah pinjaman.

```
public void angsur(int nominal) {
    jumlahPinjaman -= nominal;
    if (jumlahPinjaman < 0) {
        jumlahPinjaman = 0;
    }
}
```

b. Membuat class penguji TestKoperasi.java

- Instansiasi objek anggota1 dengan nama "Donny" dan limit pinjaman Rp5.000.000.
- Uji penolakan pinjaman saat mencoba meminjam Rp10.000.000 (melebihi limit).
- Uji peminjaman valid sebesar Rp4.000.000.
- Uji pembayaran angsuran sebesar Rp1.000.000 dan Rp3.000.000 hingga sisa pinjaman menjadi 0.

```
public class TestKoperasi {
    Run | Debug
    public static void main(String[] args) {
        Anggota anggota1 = new Anggota(nomorKtp: "111333444", nama: "Donny", limitPinjaman: 5000000);

        System.out.println("Nama Anggota: " + anggota1.getNama());
        System.out.println("Limit Pinjaman: " + anggota1.getLimitPinjaman());

        System.out.println(x: "\nMeminjam uang 10.000.000...");
        anggota1.pinjam(nominal: 10000000);
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());

        System.out.println(x: "\nMeminjam uang 4.000.000...");
        anggota1.pinjam(nominal: 4000000);
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());

        System.out.println(x: "\nMembayar angsuran 1.000.000");
        anggota1.angsur(nominal: 1000000);
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());

        System.out.println(x: "\nMembayar angsuran 3.000.000");
        anggota1.angsur(nominal: 3000000);
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());
    }
}
```

c. Output

```
D:\POLINEMA3\PBO> cmd /C ""C:\Program Files\Java\jdk-24\bin\java.exe" -XX:+$
cp C:\Users\LENOVO\AppData\Roaming\Code\User\workspaceStorage\ca968be8c98008
ws\PBO_b626801\bin TestKoperasi "
Nama Anggota: Donny
Limit Pinjaman: 5000000

Meminjam uang 10.000.000...
Maaf, jumlah pinjaman melebihi limit.
Jumlah pinjaman saat ini: 0

Meminjam uang 4.000.000...
Jumlah pinjaman saat ini: 4000000

Membayar angsuran 1.000.000
Jumlah pinjaman saat ini: 3000000

Membayar angsuran 3.000.000
Jumlah pinjaman saat ini: 0

D:\POLINEMA3\PBO>
```

2. Memodifikasi untuk penambahan validasi pada method angsur() untuk menerapkan aturan batas minimal pembayaran angsuran.
- Aturan Minimal Angsuran (10%): Pembayaran angsuran diwajibkan minimal sebesar 10% dari saldo pinjaman saat ini (jumlahPinjaman).
  - Mekanisme Pengecekan: Nilai 10% dihitung terlebih dahulu menggunakan  $0.10 * \text{jumlahPinjaman}$ . Jika nominal angsuran yang dimasukkan lebih kecil dari nilai tersebut, sistem akan menolak transaksi dan menampilkan pesan peringatan: "Maaf, angsuran harus 10% dari jumlah pinjaman"

```
public void angsur(int nominal) {
    double minimalAngsuran = 0.10 * jumlahPinjaman;
    if (nominal < minimalAngsuran) {
        System.out.println(x: "Maaf, angsuran harus 10% dari jumlah pinjaman");
    } else {
        jumlahPinjaman -= nominal;
        if (jumlahPinjaman < 0) {
            jumlahPinjaman = 0;
        }
    }
}
```

## Pengujian

```
public class TestKoperasi {  
    Run | Debug  
    public static void main(String[] args) {  
        Anggota anggota1 = new Anggota(nomorKtp: "111333444", nama: "Donny", limitPinjaman: 5000000);  
  
        System.out.println(x: "Meminjam uang 4.000.000...");  
        anggota1.pinjam(nominal: 4000000);  
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());  
  
        // mengangsur 200.000  
        System.out.println(x: "\nMembayar angsuran 200.000");  
        anggota1.angsur(nominal: 200000);  
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());  
  
        // mengangsur 1.000.000  
        System.out.println(x: "\nMembayar angsuran 1.000.000");  
        anggota1.angsur(nominal: 1000000);  
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());  
    }  
}
```

## Output

```
D:\POLINEMA3\PBO>  
D:\POLINEMA3\PBO> d: && cd d:\POLINEMA3\PBO && cmd /C ""C:\Progr  
eDetailsInExceptionMessages -cp C:\Users\LENOVO\AppData\Roaming\  
e11557ab218e\redhat.java\jdt_ws\PBO_b626801\bin TestKoperasi "  
Meminjam uang 4.000.000...  
Jumlah pinjaman saat ini: 4000000  
  
Membayar angsuran 200.000  
Maaf, angsuran harus 10% dari jumlah pinjaman  
Jumlah pinjaman saat ini: 4000000  
  
Membayar angsuran 1.000.000  
Jumlah pinjaman saat ini: 3000000  
  
d:\POLINEMA3\PBO>
```

Hasil Pengujian: Jika nominal memenuhi atau melebihi 10%, saldo jumlahPinjaman akan berkurang sesuai nilai angsuran.